SMTP TRAFFIC ANALYSIS

Detecting Phishing and Malware Delivery in Email Traffic

A packet-level investigation of a PCAP file using Wireshark

Wireshark · SMTP · PCAP analysis · Email security · Blue team

Project at a Glance

Objective	Investigate captured SMTP traffic to identify blocked mail, security rejections, and suspicious attachments.
Tool	Wireshark (display filters, packet details, Internet Message Format view)
Data	traffic.pcap: 1,637 packets in total, 512 of them SMTP; captured 19 Dec 2018
Techniques	SMTP status-code filtering, packet inspection, header analysis, attachment and encoding review
Outcome	Traced spam-list blocks, content-security rejections, an undeliverable message, and a suspicious .scr attachment sent via Outlook Express

1. Overview

Email remains one of the most common delivery channels for phishing and malware. Because the SMTP protocol exposes rich metadata (server banners, status codes, headers, and attachment details), a packet capture of mail traffic can reveal how mail servers reacted to suspicious messages and what those messages contained.

In this project I analyzed a PCAP file containing SMTP traffic with Wireshark. The goal was to answer a series of investigative questions: how mail servers greeted and rejected traffic, which messages were blocked and why, and which messages carried attachments worth a closer look.

2. Approach

1. Opened `traffic.pcap` in Wireshark and confirmed the capture size (1,637 packets).
2. Used SMTP response-code display filters (`220`, `553`, `552`) to isolate server behavior.
3. Opened individual packets to read the full response text and headers.
4. Filtered on Internet Message Format (`imf`) to inspect complete emails, including the client used and the attachment encoding.
5. Cross-checked SMTP status codes against a public reference to interpret their meaning.

3. Summary of Findings

#	Question	Filter / method	Answer
1	Packets containing SMTP response 220 (Service ready)	`smtp.response.code == 220`	**19**
2	Response code for the email blocked by spamhaus.org	`smtp.response.code == 553`	**553**
3	Full response message for that packet	`Packet details → SMTP`	**Requested action not taken: mailbox name not allowed (553)**
4	Messages blocked for presenting a potential security issue	`smtp.response.code == 552`	**6**
5	SMTP packets available for analysis	`smtp`	**512**
6	Attachment name in packet 270	`Packet 270 → packet details`	**document.zip**
7	Host IP not responding in packet 270 (undeliverable message)	`Packet 270 → message body`	**212.253.25.152**
8	Email client used to send the message with attachment.scr	`imf contains "attachment.scr"`	**Microsoft Outlook Express 6.00.2600.0000**
9	Encoding of the suspicious attachment	`Packet 685 → MIME part headers`	**base64**

4. Detailed Walkthrough

Step 1: Baseline the server greetings (220)

SMTP servers open a session with a `220 Service ready` response. Counting these gives a quick sense of how many server connections are present in the capture.

Filter: `smtp.response.code == 220`

Result: **19 packets** matched, about 1.2% of the 1,637 packets in the file. The banners include entries such as a Postfix server on localhost and Exim-based hosts.

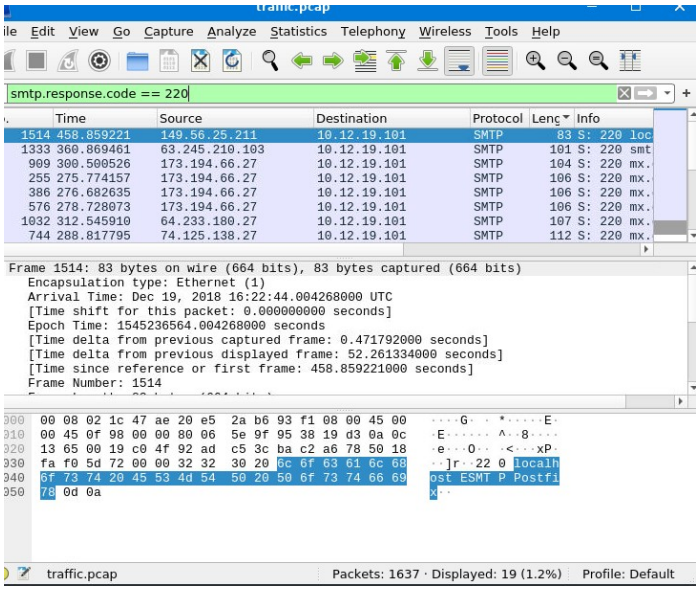


Figure 1. Filtering for SMTP response code 220 shows 19 packets (1.2% of the capture).

Step 2: Identify the message blocked by Spamhaus (553)

One server response stated that the email was blocked using spamhaus.org, a well-known spam blocklist. I used the public SMTP code reference to recognize the category, then filtered on `553` to isolate the packet.

Filter: `smtp.response.code == 553`

Result: **Code 553** in packet 156. Expanding the SMTP layer shows the full response: **Requested action not taken: mailbox name not allowed (553)**. The response text ends with the listed IP address, 173.66.46.112.

553	Requested action not taken: mailbox name not allowed	The recipient cannot be found due to errors in the email address. Check that the email addresses are correct in the To, Cc and Bcc fields.

Figure 2. Reference entry for SMTP code 553 used to interpret the response.

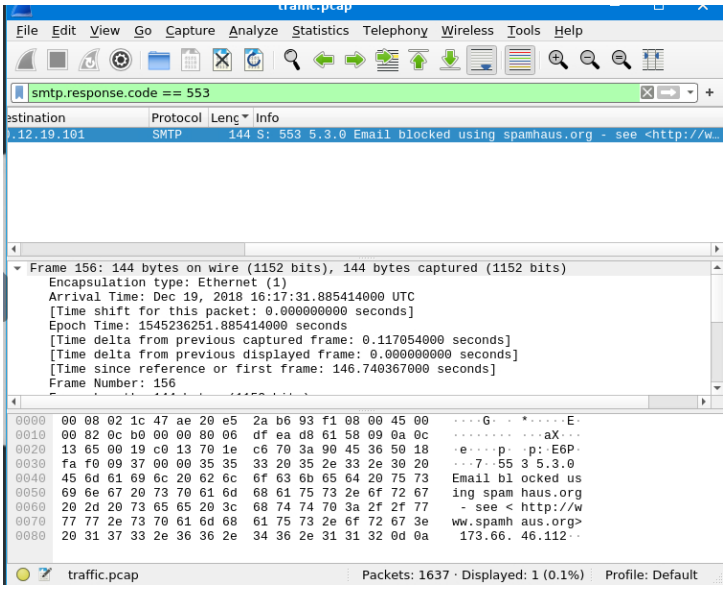


Figure 3. Filtering on 553 returns a single packet: "Email blocked using spamhaus.org".

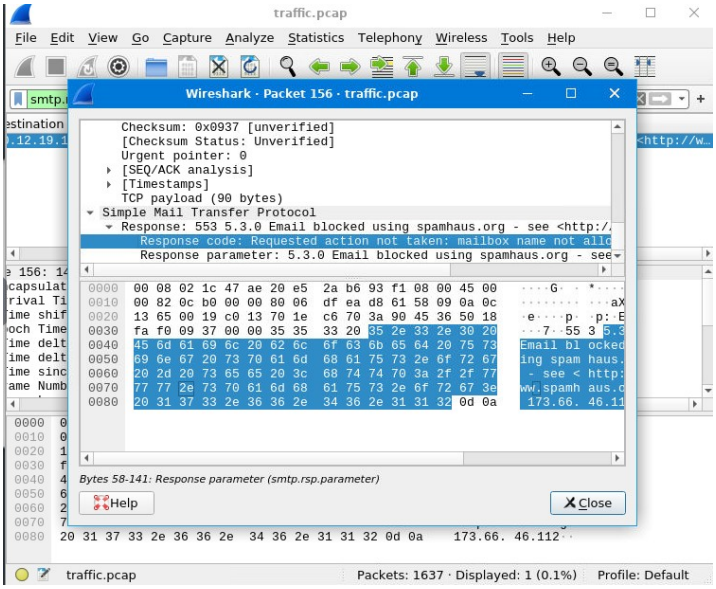


Figure 4. Expanding the SMTP layer of packet 156 reveals the full response code and parameter.

Step 3: Count messages blocked for security reasons (552)

Next I searched for code `552`. These responses read "This message was blocked because its content presents a potential security issue", meaning the receiving server rejected the content itself, not just the sender.

Filter: `smtp.response.code == 552`

Result: 6 messages were blocked. The rejections came from a small set of hosts on IP ranges consistent with Google mail infrastructure.

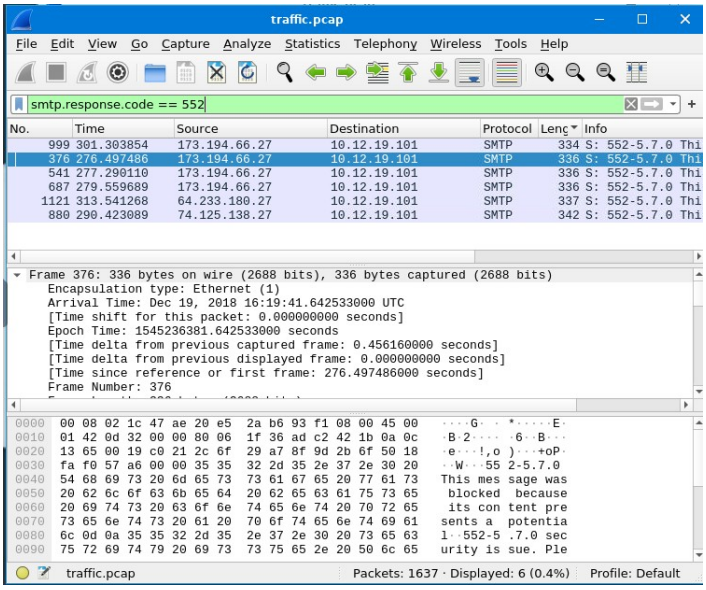


Figure 5. Six packets match SMTP response code 552.

Step 4: Size the SMTP dataset

To understand the scope of the analysis, I applied the plain `smtp` filter. Wireshark reported **512 SMTP packets**, roughly 31.3% of the capture, so nearly a third of all traffic was mail-related.

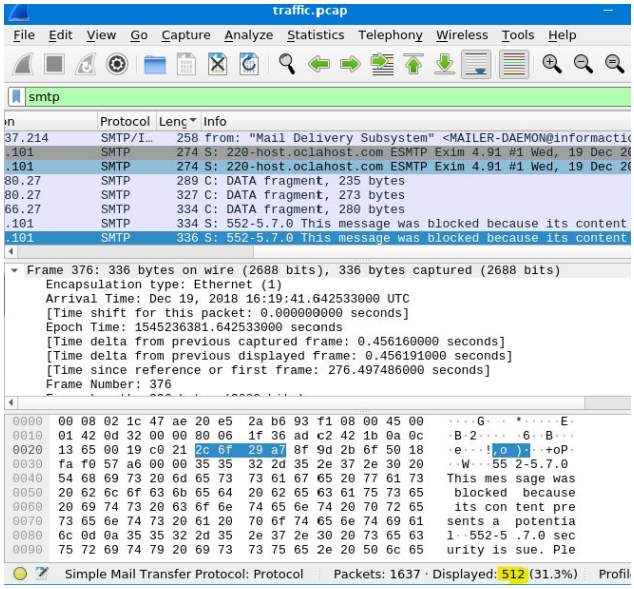


Figure 6. The smtp filter displays 512 of 1,637 packets (31.3%).

Step 5: Inspect packet 270 (attachment and undeliverable host)

Packet 270 is an email delivery report. Opening it shows two useful details: the attachment metadata, and the reason the message failed.

Attachment: The MIME part is base64-encoded and its filename is **document.zip**. Zip archives are a common way to move malicious payloads past basic filters.

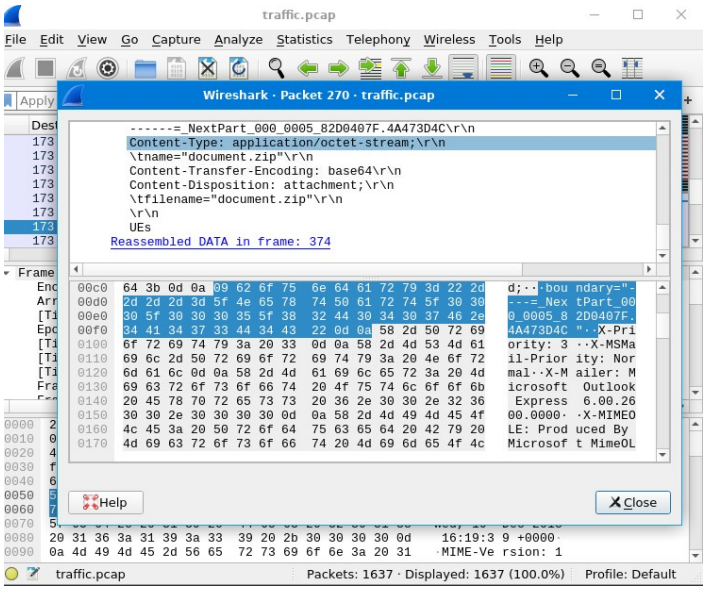


Figure 7. MIME headers in packet 270 show an octet-stream attachment named document.zip.

Undeliverable host: The body reports that the message was not delivered within 5 days because host **212.253.25.152** is not responding. The failed recipient was talkback@mozilla.org.

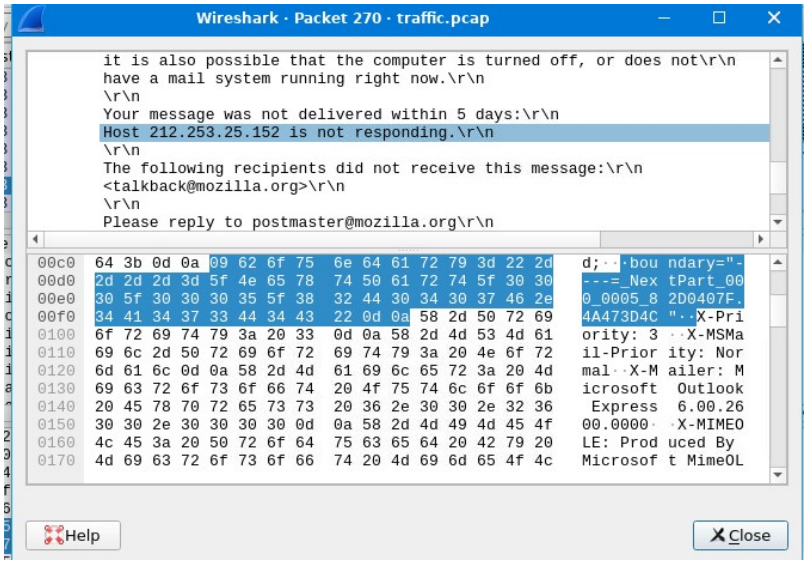


Figure 8. The delivery report names 212.253.25.152 as the host that is not responding.

Step 6: Analyze the suspicious attachment.scr

A `.scr` file is a Windows screensaver, which is really an executable. Attackers often use it to disguise malware, so I searched the capture for it using the Internet Message Format filter.

Filter: `imf contains "attachment.scr"`

Result: One message matched (packet 685): a "RETURNED MAIL: DATA FORMAT ERROR" email, made to look like a mail-system bounce, sent to inspector@mozilla.org. Bounce-style lures like this are a familiar social-engineering trick.

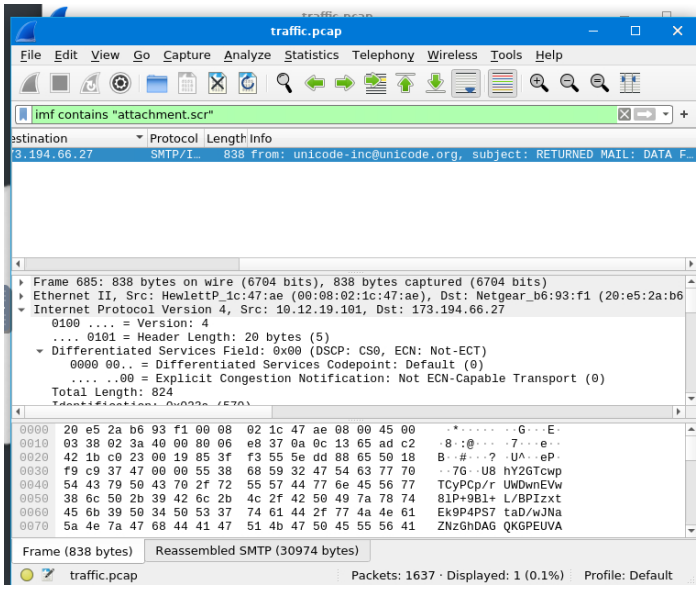


Figure 9. The imf filter isolates the email containing attachment.scr (packet 685).

Email client: The X-Mailer header shows the message was sent with **Microsoft Outlook Express 6.00.2600.0000**, a legacy client that is long out of support.

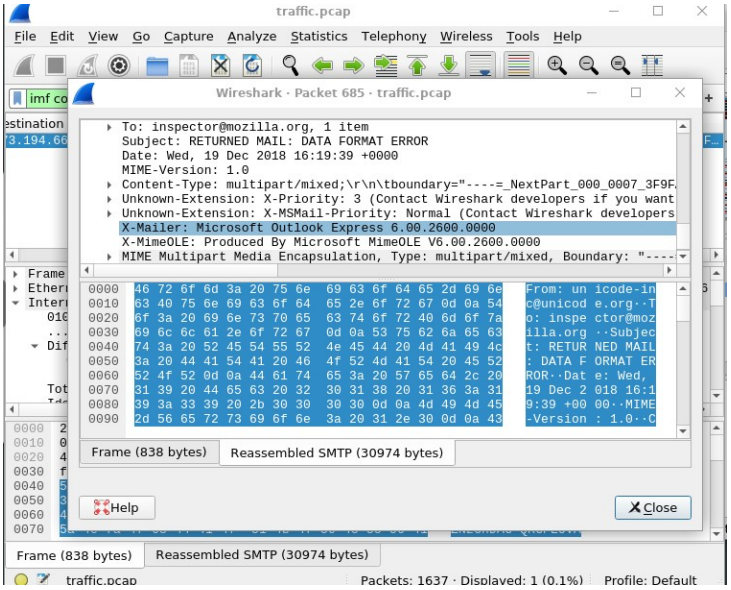


Figure 10. The X-Mailer header identifies Microsoft Outlook Express 6.00.2600.0000.

Encoding: The attachment part declares **Content-Transfer-Encoding: base64**, which converts binary content to text so it can travel through SMTP. The encoded data is about 30 KB.

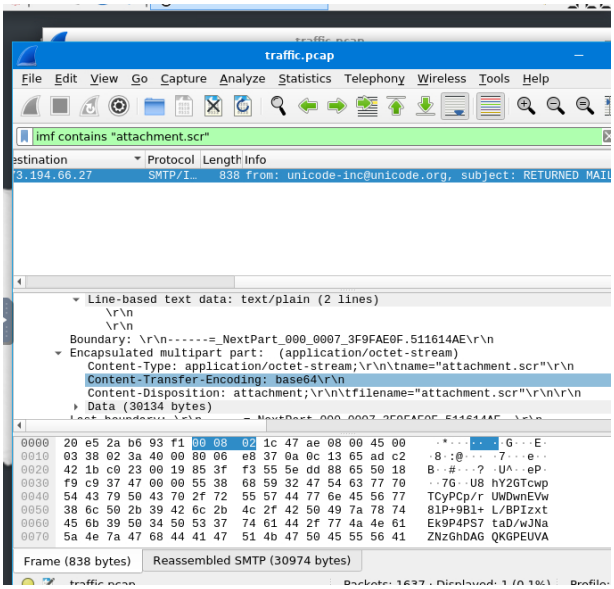


Figure 11. The attachment.scr part is base64-encoded (Content-Transfer-Encoding: base64).

5. Key Takeaways

- **Status codes tell a story.** A 553 pointed to a blocklist decision, while 552 showed content-based rejection, two different defensive layers.
- **Attachments deserve scrutiny.** Both a zip archive and a .scr executable appeared as base64 attachments, and neither is a format most legitimate email needs.
- **Bounce messages can be lures.** The "RETURNED MAIL" message carrying an executable attachment is a pattern worth flagging.
- **Header details add context.** The X-Mailer value exposed an outdated mail client, a useful clue when profiling a sender.

Note: I identified and characterized the suspicious attachment from message metadata only. I did not extract or execute its contents.

6. Defensive Recommendations

- Block or quarantine executable attachment types (.scr, .exe, .js, .vbs) at the mail gateway.
- Use sandbox detonation for archives and unknown file types before delivery.
- Subscribe to reputation blocklists such as Spamhaus to reject known-bad senders early.
- Enforce SPF, DKIM and DMARC so spoofed bounce messages are easier to detect.
- Alert on unexpected bounce or "returned mail" messages that include attachments.

7. Skills Demonstrated

- Network traffic analysis with Wireshark display filters
- SMTP protocol knowledge and status-code interpretation

- Email header and MIME structure analysis
- Recognizing phishing and malware-delivery indicators
- Clear technical documentation of an investigation

References

- MailerSend: SMTP response codes reference
- Wireshark documentation: SMTP and Internet Message Format display filters